data analysis

20 de junio de 2026

Contenido

Data Engineering Analysis - Celebal Technologies

This notebook performs comprehensive data analysis on the combined dataset including data loading, exploration, cleaning, feature engineering, analysis, and visualization.

Step 1: Load Data

Objective: Import required libraries, load the dataset, and inspect its basic structure.

Tasks:

- Import pandas, numpy, matplotlib, seaborn
- Load the combined dataset
- Check dataset shape and column names

In [2]:

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Set display options for better readability
pd.set_option('display.max_columns', None)
pd.set_option('display.max_colwidth', None)

# Set style for plots
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)

print("✓ Libraries imported successfully!")
```

Out [2]:

```
✓ Libraries imported successfully!
```

In [3]:

```
# Load the dataset
file_path = 'combined_dataset/Combined_dataset.csv'
df = pd.read_csv(file_path)
```

```
print(f"Dataset loaded successfully!")
print(f"\nDataset Shape: {df.shape}")
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")
print(f"\n{'='*50}")
print("Column Names:")
print(f"{'='*50}")
for i, col in enumerate(df.columns, 1):
    print(f"{i}. {col}")
```

Out [3]:

```
Dataset loaded successfully!

Dataset Shape: (1000, 24)
Rows: 1000, Columns: 24

==================================================
Column Names:
==================================================
1. product_id
2. title
3. product_description
4. rating
5. ratings_count
6. initial_price
7. discount
8. final_price
9. currency
10. images
11. delivery_options
12. product_details
13. breadcrumbs
14. product_specifications
15. amount_of_stars
16. what_customers_said
17. seller_name
18. sizes
19. videos
20. seller_information
21. variations
22. best_offer
23. more_offers
24. category
```

In [4]:

```
# Display first few rows
print("First 5 Rows of the Dataset:")
print("="*50)
df.head()
```

Out [4]:

```
First 5 Rows of the Dataset:
==================================================
```

	product_id	title	product_description	rating	ratings_count	initial_price	discount	final_price	currency	images	de
0	8376765	Lino Perros	Women Navy Blue Solid Backpack	3.8	15	3995	58.0	"₹3,995.00"	INR	http://assets.myntassets.com/asset…	["100% Original Prod
1	9136281	Tommy Hilfiger	Unisex Navy Blue Striped Back…	4.5	67	2899	35.0	"₹2,899.00"	INR	http://assets.myntassets.com/asset…	["100% Original Prod
2	17633752	Lavie	Aries Women Pink Mini Backpack	4.4	226	2999	65.0	"₹2,999.00"	INR	http://assets.myntassets.com/asset…	["100% Original Prod
3	1376949	F Gear	Unisex Navy & Grey Printed Bur…	4.4	1052	1675	52.0	"₹1,675.00"	INR	http://assets.myntassets.com/asset…	["100% Original Prod
4	13939916	MYTRIDENT	Men Blue Solid Bath Robe	4.7	12	2899	17.0	"₹2,899.00"	INR	http://assets.myntassets.com/asset…	["100% Original Prod

Step 2: Understand Data

Objective: Explore data types, identify missing values, and compute summary statistics.

Tasks:

- Check data types of all columns
- Identify missing/null values
- Use `.info()` and `.describe()` for summary statistics

In [5]:

```
# Check data types of all columns
print("Data Types of All Columns:")
print("="*50)
df.dtypes
```

Out [5]:

```
Data Types of All Columns:
==================================================
product_id                  int64
title                         str
product_description           str
rating                    float64
ratings_count               int64
initial_price               int64
discount                  float64
final_price                   str
currency                      str
images                        str
delivery_options              str
product_details               str
breadcrumbs                   str
product_specifications        str
amount_of_stars               str
what_customers_said           str
seller_name                   str
sizes                         str
videos                        str
seller_information            str
variations                    str
best_offer                    str
more_offers                   str
category                      str
dtype: object
```

In [6]:

```
# Check for missing values
print("\nMissing Values Analysis:")
print("="*50)
missing_values = df.isnull().sum()
missing_percentage = (missing_values / len(df)) * 100
missing_df = pd.DataFrame({
    'Column': df.columns,
    'Missing_Count': missing_values.values,
    'Missing_Percentage': missing_percentage.values
})
missing_df = missing_df[missing_df['Missing_Count'] > 0].sort_values('Missing_Count', ascending=False)
print(missing_df.to_string(index=False))
```

Out [6]:

```
Missing Values Analysis:
==================================================
             Column  Missing_Count  Missing_Percentage
             videos            781                78.1
what_customers_said            573                57.3
         variations            562                56.2
        seller_name            301                30.1
 seller_information            301                30.1
           discount            121                12.1
```

In [7]:

```
# Detailed Info about the dataset
print("\nDataset Information:")
print("="*50)
df.info()
```

Out [7]:

```
Dataset Information:
==================================================
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 24 columns):
 #   Column                  Non-Null Count  Dtype
```

```
 ---  ------                   --------------  -----
  0   product_id               1000 non-null   int64
  1   title                    1000 non-null   str
  2   product_description      1000 non-null   str
  3   rating                   1000 non-null   float64
  4   ratings_count            1000 non-null   int64
  5   initial_price            1000 non-null   int64
  6   discount                 879 non-null    float64
  7   final_price              1000 non-null   str
  8   currency                 1000 non-null   str
  9   images                   1000 non-null   str
  10  delivery_options         1000 non-null   str
  11  product_details          1000 non-null   str
  12  breadcrumbs              1000 non-null   str
  13  product_specifications   1000 non-null   str
  14  amount_of_stars          1000 non-null   str
  15  what_customers_said      427 non-null    str
  16  seller_name              699 non-null    str
  17  sizes                    1000 non-null   str
  18  videos                   219 non-null    str
  19  seller_information       699 non-null    str
  20  variations               438 non-null    str
  21  best_offer               1000 non-null   str
  22  more_offers              1000 non-null   str
  23  category                 1000 non-null   str
dtypes: float64(2), int64(3), str(19)
memory usage: 187.6 KB
```

In [8]:

```
# Summary statistics
print("\nSummary Statistics:")
print("="*50)
df.describe()
```

Out [8]:

```
Summary Statistics:
==================================================
```

	product_id	rating	ratings_count	initial_price	discount
count	1.000000e+03	1000.0000	1000.000000	1000.00000	879.000000
mean	1.713437e+07	3.6215	76.691000	2723.24100	53.503982
std	3.813766e+06	1.3744	241.114263	2408.69736	18.029201
min	5.868460e+05	0.0000	0.000000	249.00000	1.000000
25%	1.511501e+07	3.7000	7.000000	1399.00000	44.000000
50%	1.820890e+07	4.1000	17.000000	1999.00000	56.000000
75%	1.902737e+07	4.3000	58.000000	3299.00000	66.000000
max	2.274707e+07	5.0000	4441.000000	22199.00000	88.000000

Step 3: Data Cleaning

Objective: Clean the data by converting price columns to numeric, handling missing values, and removing duplicates.

Tasks:

- Convert price-related columns to numeric format
- Handle missing values appropriately
- Remove duplicate records

In [9]:

```
# Create a copy for cleaning
df_clean = df.copy()

print("Data Cleaning Process:")
print("="*50)
print(f"Original dataset shape: {df_clean.shape}")

# Step 1: Check initial_price and final_price columns
print("\nBefore Conversion:")
```

```
print(f"initial_price dtype: {df_clean['initial_price'].dtype}")
print(f"final_price dtype: {df_clean['final_price'].dtype}")
print(f"Sample initial_price values: {df_clean['initial_price'].head()}")
print(f"Sample final_price values: {df_clean['final_price'].head()}")
```

Out [9]:

```
Data Cleaning Process:
==================================================
Original dataset shape: (1000, 24)

Before Conversion:
initial_price dtype: int64
final_price dtype: str
Sample initial_price values: 0    3995
1    2899
2    2999
3    1675
4    2899
Name: initial_price, dtype: int64
Sample final_price values: 0    "₹3,995.00"
1    "₹2,899.00"
2    "₹2,999.00"
3    "₹1,675.00"
4    "₹2,899.00"
Name: final_price, dtype: str
```

In [10]:

```
# Convert price columns to numeric
# Remove any non-numeric characters and convert to float
df_clean['initial_price'] = pd.to_numeric(df_clean['initial_price'], errors='coerce')
df_clean['final_price'] = df_clean['final_price'].astype(str).str.replace('₹', '').str.replace(',', '').str.replace('"', '')
df_clean['final_price'] = pd.to_numeric(df_clean['final_price'], errors='coerce')
df_clean['discount'] = pd.to_numeric(df_clean['discount'], errors='coerce')
df_clean['rating'] = pd.to_numeric(df_clean['rating'], errors='coerce')
df_clean['ratings_count'] = pd.to_numeric(df_clean['ratings_count'], errors='coerce')

print("\nAfter Conversion:")
print(f"initial_price dtype: {df_clean['initial_price'].dtype}")
print(f"final_price dtype: {df_clean['final_price'].dtype}")
print(f"Sample initial_price values: {df_clean['initial_price'].head()}")
print(f"Sample final_price values: {df_clean['final_price'].head()}")
```

Out [10]:

```
After Conversion:
initial_price dtype: int64
final_price dtype: float64
Sample initial_price values: 0    3995
1    2899
2    2999
3    1675
4    2899
Name: initial_price, dtype: int64
Sample final_price values: 0    3995.0
1    2899.0
2    2999.0
3    1675.0
4    2899.0
Name: final_price, dtype: float64
```

In [11]:

```
# Handle missing values
print("\nHandling Missing Values:")
print("="*50)
print(f"Before: {df_clean.isnull().sum().sum()} total missing values")

# Fill missing values for numeric columns with median
numeric_columns = ['initial_price', 'final_price', 'discount', 'rating', 'ratings_count']
for col in numeric_columns:
    if df_clean[col].isnull().sum() > 0:
        df_clean[col].fillna(df_clean[col].median(), inplace=True)

print(f"After: {df_clean.isnull().sum().sum()} total missing values")
```

```
# Remove rows with missing values in critical columns
df_clean = df_clean.dropna(subset=['product_id', 'title', 'category'])

print(f"Dataset shape after handling missing values: {df_clean.shape}")
```

Out [11]:

```
Handling Missing Values:
==================================================
Before: 2639 total missing values
After: 2639 total missing values
Dataset shape after handling missing values: (1000, 24)
```

In [12]:

```
# Remove duplicate records
print("\nRemoving Duplicate Records:")
print("="*50)
print(f"Before: {len(df_clean)} rows")

# Remove duplicates based on product_id
df_clean = df_clean.drop_duplicates(subset=['product_id'], keep='first')

print(f"After removing duplicates: {len(df_clean)} rows")
print(f"Duplicates removed: {df.shape[0] - df_clean.shape[0]} rows")
print(f"\n✓ Data Cleaning Complete!")
print(f"Final dataset shape: {df_clean.shape}")
```

Out [12]:

```
Removing Duplicate Records:
==================================================
Before: 1000 rows
After removing duplicates: 1000 rows
Duplicates removed: 0 rows

✓ Data Cleaning Complete!
Final dataset shape: (1000, 24)
```

Step 4: Feature Engineering

Objective: Create new features to enhance analysis.

Tasks:

- Create a price difference column (initial price vs final price)
- Create a popularity metric (using rating and ratings count)
- Extract useful information from complex fields

In [13]:

```
# Feature Engineering
print("Feature Engineering:")
print("="*50)

# 1. Price Difference Column
df_clean['price_difference'] = df_clean['initial_price'] - df_clean['final_price']
df_clean['price_diff_percentage'] = (df_clean['price_difference'] / df_clean['initial_price'] * 100).round(2)

print("\n1. Price Difference Features Created:")
print(f"   - price_difference: Absolute difference")
print(f"   - price_diff_percentage: Percentage difference")
print(f"\n   Sample values:")
print(df_clean[['initial_price', 'final_price', 'price_difference', 'price_diff_percentage']].head())
```

Out [13]:

```
Feature Engineering:
==================================================

1. Price Difference Features Created:
   - price_difference: Absolute difference
   - price_diff_percentage: Percentage difference

   Sample values:
```

```
   initial_price  final_price  price_difference  price_diff_percentage
0           3995       3995.0               0.0                    0.0
1           2899       2899.0               0.0                    0.0
2           2999       2999.0               0.0                    0.0
3           1675       1675.0               0.0                    0.0
4           2899       2899.0               0.0                    0.0
```

In [15]:

```
# 2. Popularity Metric
# Normalize rating and ratings_count to create a composite popularity score
# popularity_metric = (normalized_rating * 0.4) + (normalized_ratings_count * 0.6)

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df_clean['rating_normalized'] = scaler.fit_transform(df_clean[['rating']])
df_clean['ratings_count_normalized'] = scaler.fit_transform(df_clean[['ratings_count']])

# Create popularity metric (weighted combination)
df_clean['popularity_metric'] = (df_clean['rating_normalized'] * 0.4 +
                                 df_clean['ratings_count_normalized'] * 0.6).round(3)

print("\n2. Popularity Metric Created:")
print(f"   Formula: (normalized_rating * 0.4) + (normalized_ratings_count * 0.6)")
print(f"   Range: 0 to 1 (higher = more popular)")
print(f"\n   Sample values:")
print(df_clean[['rating', 'ratings_count', 'popularity_metric']].head(10))
```

Out [15]:

```
2. Popularity Metric Created:
   Formula: (normalized_rating * 0.4) + (normalized_ratings_count * 0.6)
   Range: 0 to 1 (higher = more popular)

   Sample values:
   rating  ratings_count  popularity_metric
0     3.8             15              0.306
1     4.5             67              0.369
2     4.4            226              0.383
3     4.4           1052              0.494
4     4.7             12              0.378
5     4.5             42              0.366
6     4.0              5              0.321
7     4.7             10              0.377
8     4.5             22              0.363
9     3.7              7              0.297
```

In [16]:

```
# 3. Extract category information (already available, but let's categorize price ranges)
df_clean['price_range'] = pd.cut(df_clean['final_price'],
                                 bins=[0, 500, 1000, 2000, 5000, 10000, float('inf')],
                                 labels=['Budget', 'Economy', 'Mid-Range', 'Premium', 'Luxury', 'Ultra-Luxury'])

# Discount category
df_clean['discount_category'] = pd.cut(df_clean['discount'],
                                       bins=[-1, 10, 20, 30, 50, 100],
                                       labels=['Low', 'Medium', 'High', 'Very High', 'Extreme'])

print("\n3. Additional Features Created:")
print(f"   - price_range: Categorized price brackets")
print(f"   - discount_category: Categorized discount levels")
print(f"\n   Distribution of price_range:")
print(df_clean['price_range'].value_counts().sort_index())
print(f"\n   Distribution of discount_category:")
print(df_clean['discount_category'].value_counts().sort_index())
```

Out [16]:

```
3. Additional Features Created:
   - price_range: Categorized price brackets
   - discount_category: Categorized discount levels

   Distribution of price_range:
price_range
```

```
Budget          152
Economy         324
Mid-Range       267
Premium         217
Luxury           31
Ultra-Luxury      9
Name: count, dtype: int64

   Distribution of discount_category:
discount_category
Low           25
Medium        34
High          67
Very High    230
Extreme      523
Name: count, dtype: int64
```

In [17]:

```
# Display all new features
print("\n" + "="*50)
print("✓ Feature Engineering Complete!")
print("="*50)
print("\nNew Features Summary:")
print(df_clean[['price_difference', 'price_diff_percentage', 'popularity_metric',
                'price_range', 'discount_category']].describe())
```

Out [17]:

```
==================================================
✓ Feature Engineering Complete!
==================================================

New Features Summary:
       price_difference  price_diff_percentage  popularity_metric
count       1000.000000            1000.000000        1000.000000
mean        1017.145000              34.914920           0.300081
std         1393.966867              29.869129           0.118541
min            0.000000               0.000000           0.000000
25%            0.000000               0.000000           0.300000
50%          659.000000              40.040000           0.337000
75%         1476.250000              62.020000           0.361000
max        14652.000000              88.020000           0.960000
```

Step 5: Analysis

Objective: Perform comprehensive analysis including univariate, bivariate, and category-level insights.

Tasks:

- Univariate Analysis: Analyze single variables
- Bivariate Analysis: Examine relationships between variables
- Category-level Analysis: Deep insights by product category

In [18]:

```
# UNIVARIATE ANALYSIS
print("="*70)
print("UNIVARIATE ANALYSIS: Single Variable Analysis")
print("="*70)

# Analysis of Rating
print("\n1. RATING ANALYSIS:")
print(f"   Mean Rating: {df_clean['rating'].mean():.2f}")
print(f"   Median Rating: {df_clean['rating'].median():.2f}")
print(f"   Std Dev: {df_clean['rating'].std():.2f}")
print(f"   Min: {df_clean['rating'].min():.2f}, Max: {df_clean['rating'].max():.2f}")
print(f"   Rating Distribution:")
print(df_clean['rating'].value_counts().head(10).sort_index())

# Analysis of Ratings Count
print("\n2. RATINGS COUNT ANALYSIS:")
print(f"   Mean Reviews: {df_clean['ratings_count'].mean():.0f}")
print(f"   Median Reviews: {df_clean['ratings_count'].median():.0f}")
```

```
print(f"   Std Dev: {df_clean['ratings_count'].std():.0f}")

# Analysis of Final Price
print("\n3. FINAL PRICE ANALYSIS:")
print(f"   Mean Price: ₹{df_clean['final_price'].mean():.2f}")
print(f"   Median Price: ₹{df_clean['final_price'].median():.2f}")
print(f"   Std Dev: ₹{df_clean['final_price'].std():.2f}")
print(f"   Min: ₹{df_clean['final_price'].min():.2f}, Max: ₹{df_clean['final_price'].max():.2f}")

# Analysis of Discount
print("\n4. DISCOUNT ANALYSIS:")
print(f"   Mean Discount: {df_clean['discount'].mean():.2f}%")
print(f"   Median Discount: {df_clean['discount'].median():.2f}%")
print(f"   Std Dev: {df_clean['discount'].std():.2f}%")
print(f"   Min: {df_clean['discount'].min():.2f}%, Max: {df_clean['discount'].max():.2f}%")

# Analysis of Price Difference
print("\n5. PRICE DIFFERENCE ANALYSIS:")
print(f"   Mean Savings: ₹{df_clean['price_difference'].mean():.2f}")
print(f"   Median Savings: ₹{df_clean['price_difference'].median():.2f}")
print(f"   Total Potential Savings: ₹{df_clean['price_difference'].sum():.0f}")
```

Out [18]:

```
======================================================================
UNIVARIATE ANALYSIS: Single Variable Analysis
======================================================================

1. RATING ANALYSIS:
   Mean Rating: 3.62
   Median Rating: 4.10
   Std Dev: 1.37
   Min: 0.00, Max: 5.00
   Rating Distribution:
rating
0.0    114
3.7     44
3.8     49
3.9     64
4.0     82
4.1     90
4.2     90
4.3    105
4.4     83
4.5     61
Name: count, dtype: int64

2. RATINGS COUNT ANALYSIS:
   Mean Reviews: 77
   Median Reviews: 17
   Std Dev: 241

3. FINAL PRICE ANALYSIS:
   Mean Price: ₹1706.10
   Median Price: ₹1099.00
   Std Dev: ₹1783.86
   Min: ₹199.00, Max: ₹17995.00

4. DISCOUNT ANALYSIS:
   Mean Discount: 53.50%
   Median Discount: 56.00%
   Std Dev: 18.03%
   Min: 1.00%, Max: 88.00%

5. PRICE DIFFERENCE ANALYSIS:
   Mean Savings: ₹1017.14
   Median Savings: ₹659.00
   Total Potential Savings: ₹1017145
```

In [19]:

```
# BIVARIATE ANALYSIS
print("\n" + "="*70)
print("BIVARIATE ANALYSIS: Relationship between Variables")
print("="*70)
```

```
# 1. Rating vs Ratings Count correlation
correlation = df_clean['rating'].corr(df_clean['ratings_count'])
print(f"\n1. Rating vs Ratings Count Correlation: {correlation:.4f}")

# 2. Final Price vs Rating correlation
correlation = df_clean['final_price'].corr(df_clean['rating'])
print(f"2. Final Price vs Rating Correlation: {correlation:.4f}")

# 3. Discount vs Ratings Count correlation
correlation = df_clean['discount'].corr(df_clean['ratings_count'])
print(f"3. Discount vs Ratings Count Correlation: {correlation:.4f}")

# 4. Price Difference vs Rating correlation
correlation = df_clean['price_difference'].corr(df_clean['rating'])
print(f"4. Price Difference vs Rating Correlation: {correlation:.4f}")

# 5. Popularity Metric vs Final Price
correlation = df_clean['popularity_metric'].corr(df_clean['final_price'])
print(f"5. Popularity Metric vs Final Price Correlation: {correlation:.4f}")

# Create correlation matrix for key numeric variables
numeric_features = ['rating', 'ratings_count', 'final_price', 'discount',
                    'price_difference', 'popularity_metric']
correlation_matrix = df_clean[numeric_features].corr()
print(f"\n\nCorrelation Matrix of Key Features:")
print(correlation_matrix)
```

Out [19]:

```
======================================================================
BIVARIATE ANALYSIS: Relationship between Variables
======================================================================

1. Rating vs Ratings Count Correlation: 0.1258
2. Final Price vs Rating Correlation: 0.0846
3. Discount vs Ratings Count Correlation: 0.0434
4. Price Difference vs Rating Correlation: -0.2491
5. Popularity Metric vs Final Price Correlation: 0.0701


Correlation Matrix of Key Features:
                     rating  ratings_count  final_price  discount  \
rating             1.000000       0.125795     0.084587 -0.135823
ratings_count      0.125795       1.000000    -0.030233  0.043431
final_price        0.084587      -0.030233     1.000000 -0.067643
discount          -0.135823       0.043431    -0.067643  1.000000
price_difference  -0.249050      -0.001261     0.136028  0.301049
popularity_metric  0.962129       0.391452     0.070108 -0.114457

                   price_difference  popularity_metric
rating                   -0.249050           0.962129
ratings_count            -0.001261           0.391452
final_price               0.136028           0.070108
discount                  0.301049          -0.114457
price_difference          1.000000          -0.231489
popularity_metric        -0.231489           1.000000
```

In [20]:

```
# CATEGORY-LEVEL ANALYSIS
print("\n" + "="*70)
print("CATEGORY-LEVEL ANALYSIS: Insights by Product Category")
print("="*70)

category_analysis = df_clean.groupby('category').agg({
    'final_price': ['mean', 'median', 'min', 'max'],
    'rating': ['mean', 'count'],
    'discount': 'mean',
    'ratings_count': 'mean',
    'popularity_metric': 'mean'
}).round(2)

category_analysis.columns = ['Avg_Price', 'Median_Price', 'Min_Price', 'Max_Price',
                             'Avg_Rating', 'Product_Count', 'Avg_Discount',
                             'Avg_Reviews', 'Avg_Popularity']
```

```
print("\nTop 10 Categories by Product Count:")
top_categories = df_clean['category'].value_counts().head(10)
print(top_categories)

print("\nCategory Performance Summary:")
print(category_analysis.sort_values('Avg_Popularity', ascending=False).head(10))
```

Out [20]:

```
======================================================================
CATEGORY-LEVEL ANALYSIS: Insights by Product Category
======================================================================

Top 10 Categories by Product Count:
category
tops            122
dresses         100
shirts           97
jeans            57
sports-shoes     51
tshirts          39
earrings         34
sweaters         34
jackets          29
casual-shoes     23
Name: count, dtype: int64

Category Performance Summary:
                      Avg_Price  Median_Price  Min_Price  Max_Price  \
category
boots                    2463.0        2463.0     1087.0     3839.0
tunics                    685.5         683.0      636.0      740.0
cutlery                  1750.0        1750.0     1750.0     1750.0
shaving-essentials        285.0         285.0      285.0      285.0
backpacks                2892.0        2949.0     1675.0     3995.0
face-wash-and-cleanser    426.0         426.0      426.0      426.0
face-moisturisers         478.0         478.0      280.0      676.0
chair-cover               935.0         935.0      935.0      935.0
bath-robe                2899.0        2899.0     2899.0     2899.0
clothing-set              516.5         516.5      314.0      719.0

                      Avg_Rating  Product_Count  Avg_Discount  Avg_Reviews  \
category
boots                       4.45              2          74.0        865.5
tunics                      4.28              4          45.0        470.5
cutlery                     5.00              1           NaN          5.0
shaving-essentials          4.90              1           3.0          7.0
backpacks                   4.28              4          52.5        340.0
face-wash-and-cleanser      4.80              1           5.0          9.0
face-moisturisers           4.00              2          16.0        456.0
chair-cover                 4.70              1          64.0         15.0
bath-robe                   4.70              1          17.0         12.0
clothing-set                4.30              2          50.0        233.5

                      Avg_Popularity
category
boots                           0.47
tunics                          0.41
cutlery                         0.40
shaving-essentials              0.39
backpacks                       0.39
face-wash-and-cleanser          0.38
face-moisturisers               0.38
chair-cover                     0.38
bath-robe                       0.38
clothing-set                    0.38
```

In [21]:

```
# Insights by Price Range
print("\n" + "-"*70)
print("ANALYSIS BY PRICE RANGE:")
print("-"*70)

price_range_analysis = df_clean.groupby('price_range').agg({
    'final_price': ['mean', 'count'],
    'rating': 'mean',
```

```
    'discount': 'mean',
    'ratings_count': 'mean',
    'popularity_metric': 'mean'
}).round(2)

print("\nPrice Range Performance:")
print(price_range_analysis)

# Insights by Discount Category
print("\n" + "-"*70)
print("ANALYSIS BY DISCOUNT CATEGORY:")
print("-"*70)

discount_analysis = df_clean.groupby('discount_category').agg({
    'final_price': 'mean',
    'rating': 'mean',
    'product_id': 'count',
    'ratings_count': 'mean',
    'popularity_metric': 'mean'
}).round(2)

discount_analysis.columns = ['Avg_Price', 'Avg_Rating', 'Product_Count', 'Avg_Reviews', 'Avg_Popularity']
print("\nDiscount Category Performance:")
print(discount_analysis)
```

Out [21]:

```
----------------------------------------------------------------------
ANALYSIS BY PRICE RANGE:
----------------------------------------------------------------------

Price Range Performance:
             final_price       rating discount ratings_count popularity_metric
                    mean count   mean     mean          mean              mean
price_range
Budget            391.11   152   3.66    55.00         65.22              0.30
Economy           752.53   324   3.32    55.06         81.78              0.28
Mid-Range        1501.75   267   3.76    52.41         85.60              0.31
Premium          3129.50   217   3.84    51.47         72.32              0.32
Luxury           6661.71    31   3.76    51.05         50.29              0.31
Ultra-Luxury    12916.11     9   4.06    47.00         19.11              0.33

----------------------------------------------------------------------
ANALYSIS BY DISCOUNT CATEGORY:
----------------------------------------------------------------------

Discount Category Performance:
                  Avg_Price  Avg_Rating  Product_Count  Avg_Reviews  \
discount_category
Low                 1113.48        3.82             25        39.60
Medium              1216.91        3.91             34        59.12
High                1874.07        3.75             67        42.49
Very High           1964.08        3.75            230        67.88
Extreme             1378.53        3.41            523        83.46

                  Avg_Popularity
discount_category
Low                         0.31
Medium                      0.32
High                        0.31
Very High                   0.31
Extreme                     0.28
```

Step 6: Visualization

Objective: Create comprehensive visualizations to represent trends and patterns.

Plots:

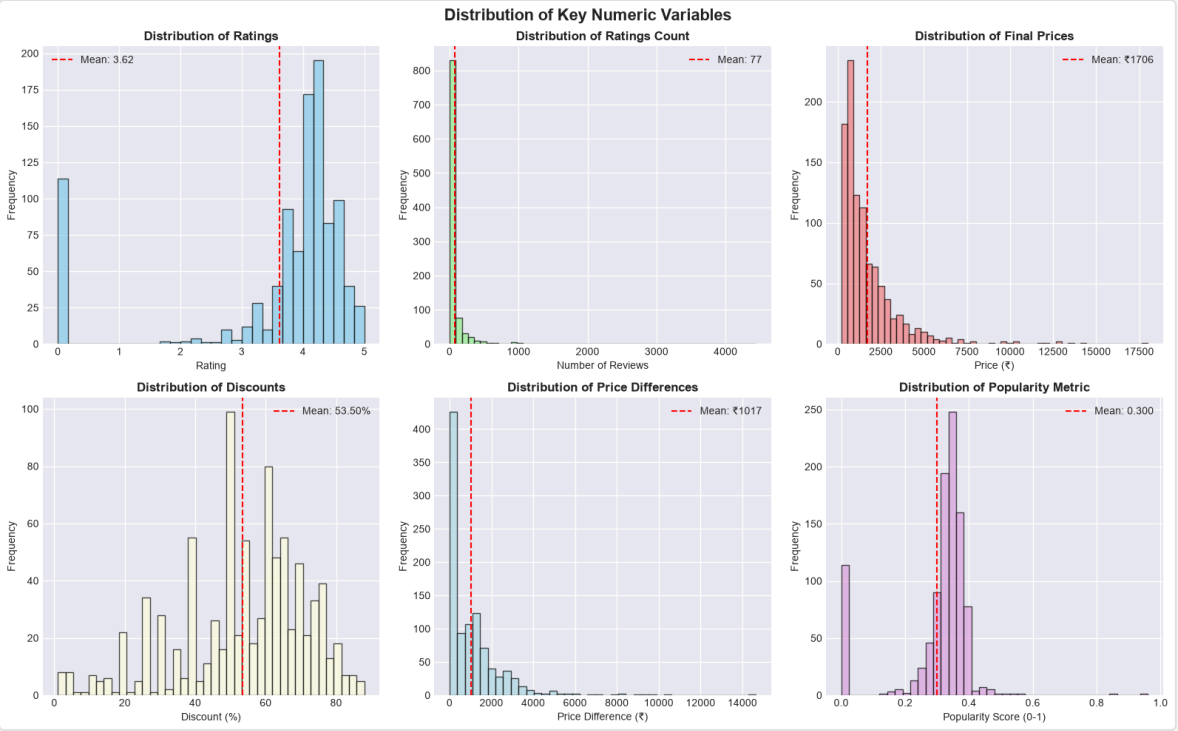
- Histograms: Distribution of numeric variables
- Bar Charts: Categorical analysis
- Boxplots: Outlier detection and distribution comparison

In [22]:

```
# VISUALIZATION
print("="*70)
print("CREATING VISUALIZATIONS")
print("="*70)

# Set up the style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# 1. HISTOGRAMS - Distribution of Numeric Variables
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Distribution of Key Numeric Variables', fontsize=16, fontweight='bold')

# Rating Distribution
axes[0, 0].hist(df_clean['rating'], bins=30, color='skyblue', edgecolor='black', alpha=0.7)
axes[0, 0].set_title('Distribution of Ratings', fontweight='bold')
axes[0, 0].set_xlabel('Rating')
axes[0, 0].set_ylabel('Frequency')
axes[0, 0].axvline(df_clean['rating'].mean(), color='red', linestyle='--', label=f'Mean: {df_clean["rating"].mean():.2f}')
axes[0, 0].legend()

# Ratings Count Distribution
axes[0, 1].hist(df_clean['ratings_count'], bins=50, color='lightgreen', edgecolor='black', alpha=0.7)
axes[0, 1].set_title('Distribution of Ratings Count', fontweight='bold')
axes[0, 1].set_xlabel('Number of Reviews')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].axvline(df_clean['ratings_count'].mean(), color='red', linestyle='--', label=f'Mean: {df_clean["ratings_count"].mean():
axes[0, 1].legend()

# Final Price Distribution
axes[0, 2].hist(df_clean['final_price'], bins=50, color='lightcoral', edgecolor='black', alpha=0.7)
axes[0, 2].set_title('Distribution of Final Prices', fontweight='bold')
axes[0, 2].set_xlabel('Price (₹)')
axes[0, 2].set_ylabel('Frequency')
axes[0, 2].axvline(df_clean['final_price'].mean(), color='red', linestyle='--', label=f'Mean: ₹{df_clean["final_price"].mean():.0f
axes[0, 2].legend()

# Discount Distribution
axes[1, 0].hist(df_clean['discount'], bins=40, color='lightyellow', edgecolor='black', alpha=0.7)
axes[1, 0].set_title('Distribution of Discounts', fontweight='bold')
axes[1, 0].set_xlabel('Discount (%)')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].axvline(df_clean['discount'].mean(), color='red', linestyle='--', label=f'Mean: {df_clean["discount"].mean():.2f}%')
axes[1, 0].legend()

# Price Difference Distribution
axes[1, 1].hist(df_clean['price_difference'], bins=40, color='lightblue', edgecolor='black', alpha=0.7)
axes[1, 1].set_title('Distribution of Price Differences', fontweight='bold')
axes[1, 1].set_xlabel('Price Difference (₹)')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].axvline(df_clean['price_difference'].mean(), color='red', linestyle='--', label=f'Mean: ₹{df_clean["price_difference"].
axes[1, 1].legend()

# Popularity Metric Distribution
axes[1, 2].hist(df_clean['popularity_metric'], bins=40, color='plum', edgecolor='black', alpha=0.7)
axes[1, 2].set_title('Distribution of Popularity Metric', fontweight='bold')
axes[1, 2].set_xlabel('Popularity Score (0-1)')
axes[1, 2].set_ylabel('Frequency')
axes[1, 2].axvline(df_clean['popularity_metric'].mean(), color='red', linestyle='--', label=f'Mean: {df_clean["popularity_metric"]
axes[1, 2].legend()

plt.tight_layout()
plt.show()
print("✓ Histograms created successfully!")
```

Out [22]:

```
======================================================================
CREATING VISUALIZATIONS
======================================================================
```

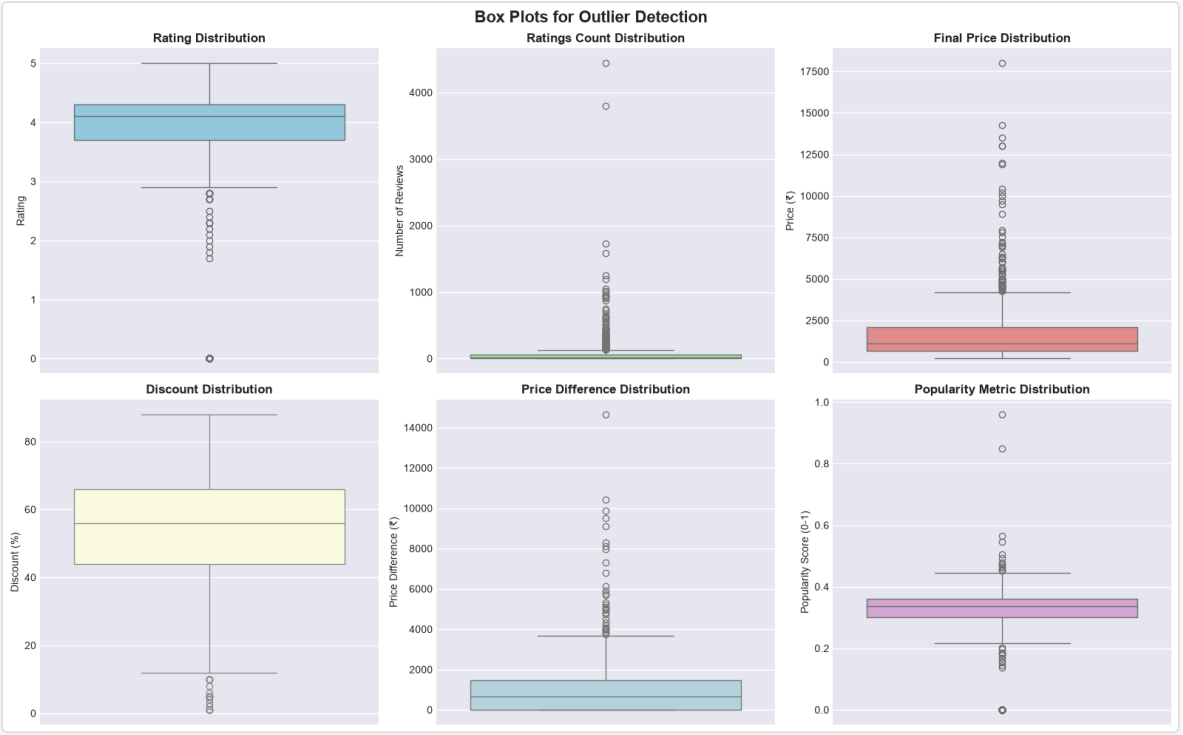


✓ Histograms created successfully!

In [23]:

```
# 2. BOX PLOTS - Detecting Outliers and Distribution Patterns
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Box Plots for Outlier Detection', fontsize=16, fontweight='bold')

# Rating Boxplot
sns.boxplot(y=df_clean['rating'], ax=axes[0, 0], color='skyblue')
axes[0, 0].set_title('Rating Distribution', fontweight='bold')
axes[0, 0].set_ylabel('Rating')

# Ratings Count Boxplot
sns.boxplot(y=df_clean['ratings_count'], ax=axes[0, 1], color='lightgreen')
axes[0, 1].set_title('Ratings Count Distribution', fontweight='bold')
axes[0, 1].set_ylabel('Number of Reviews')

# Final Price Boxplot
sns.boxplot(y=df_clean['final_price'], ax=axes[0, 2], color='lightcoral')
axes[0, 2].set_title('Final Price Distribution', fontweight='bold')
axes[0, 2].set_ylabel('Price (₹)')

# Discount Boxplot
sns.boxplot(y=df_clean['discount'], ax=axes[1, 0], color='lightyellow')
axes[1, 0].set_title('Discount Distribution', fontweight='bold')
axes[1, 0].set_ylabel('Discount (%)')

# Price Difference Boxplot
sns.boxplot(y=df_clean['price_difference'], ax=axes[1, 1], color='lightblue')
axes[1, 1].set_title('Price Difference Distribution', fontweight='bold')
axes[1, 1].set_ylabel('Price Difference (₹)')

# Popularity Metric Boxplot
sns.boxplot(y=df_clean['popularity_metric'], ax=axes[1, 2], color='plum')
axes[1, 2].set_title('Popularity Metric Distribution', fontweight='bold')
axes[1, 2].set_ylabel('Popularity Score (0-1)')

plt.tight_layout()
plt.show()
print("✓ Box plots created successfully!")
```

Out [23]:



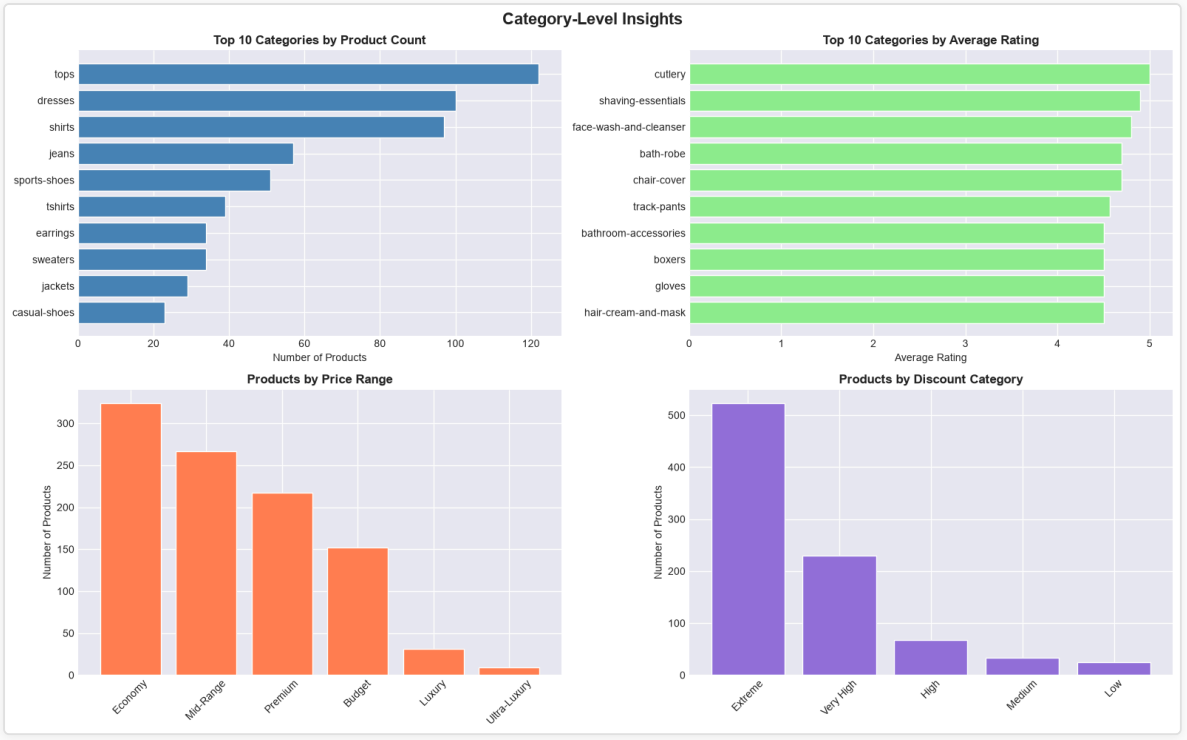
✓ Box plots created successfully!

In [24]:

```
# 3. BAR CHARTS - Categorical Analysis

# Top 10 Categories by Product Count
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle('Category-Level Insights', fontsize=16, fontweight='bold')

# Top Categories by Product Count
top_categories = df_clean['category'].value_counts().head(10)
axes[0, 0].barh(range(len(top_categories)), top_categories.values, color='steelblue')
axes[0, 0].set_yticks(range(len(top_categories)))
axes[0, 0].set_yticklabels(top_categories.index)
axes[0, 0].set_title('Top 10 Categories by Product Count', fontweight='bold')
axes[0, 0].set_xlabel('Number of Products')
axes[0, 0].invert_yaxis()

# Top Categories by Average Rating
top_categories_rating = df_clean.groupby('category')['rating'].mean().nlargest(10)
axes[0, 1].barh(range(len(top_categories_rating)), top_categories_rating.values, color='lightgreen')
axes[0, 1].set_yticks(range(len(top_categories_rating)))
axes[0, 1].set_yticklabels(top_categories_rating.index)
axes[0, 1].set_title('Top 10 Categories by Average Rating', fontweight='bold')
axes[0, 1].set_xlabel('Average Rating')
axes[0, 1].invert_yaxis()

# Distribution by Price Range
price_range_counts = df_clean['price_range'].value_counts()
axes[1, 0].bar(range(len(price_range_counts)), price_range_counts.values, color='coral')
axes[1, 0].set_xticks(range(len(price_range_counts)))
axes[1, 0].set_xticklabels(price_range_counts.index, rotation=45)
axes[1, 0].set_title('Products by Price Range', fontweight='bold')
axes[1, 0].set_ylabel('Number of Products')

# Distribution by Discount Category
discount_counts = df_clean['discount_category'].value_counts()
axes[1, 1].bar(range(len(discount_counts)), discount_counts.values, color='mediumpurple')
axes[1, 1].set_xticks(range(len(discount_counts)))
axes[1, 1].set_xticklabels(discount_counts.index, rotation=45)
axes[1, 1].set_title('Products by Discount Category', fontweight='bold')
axes[1, 1].set_ylabel('Number of Products')
```

```
plt.tight_layout()
plt.show()
print("✓ Category bar charts created successfully!")
```

Out [24]:

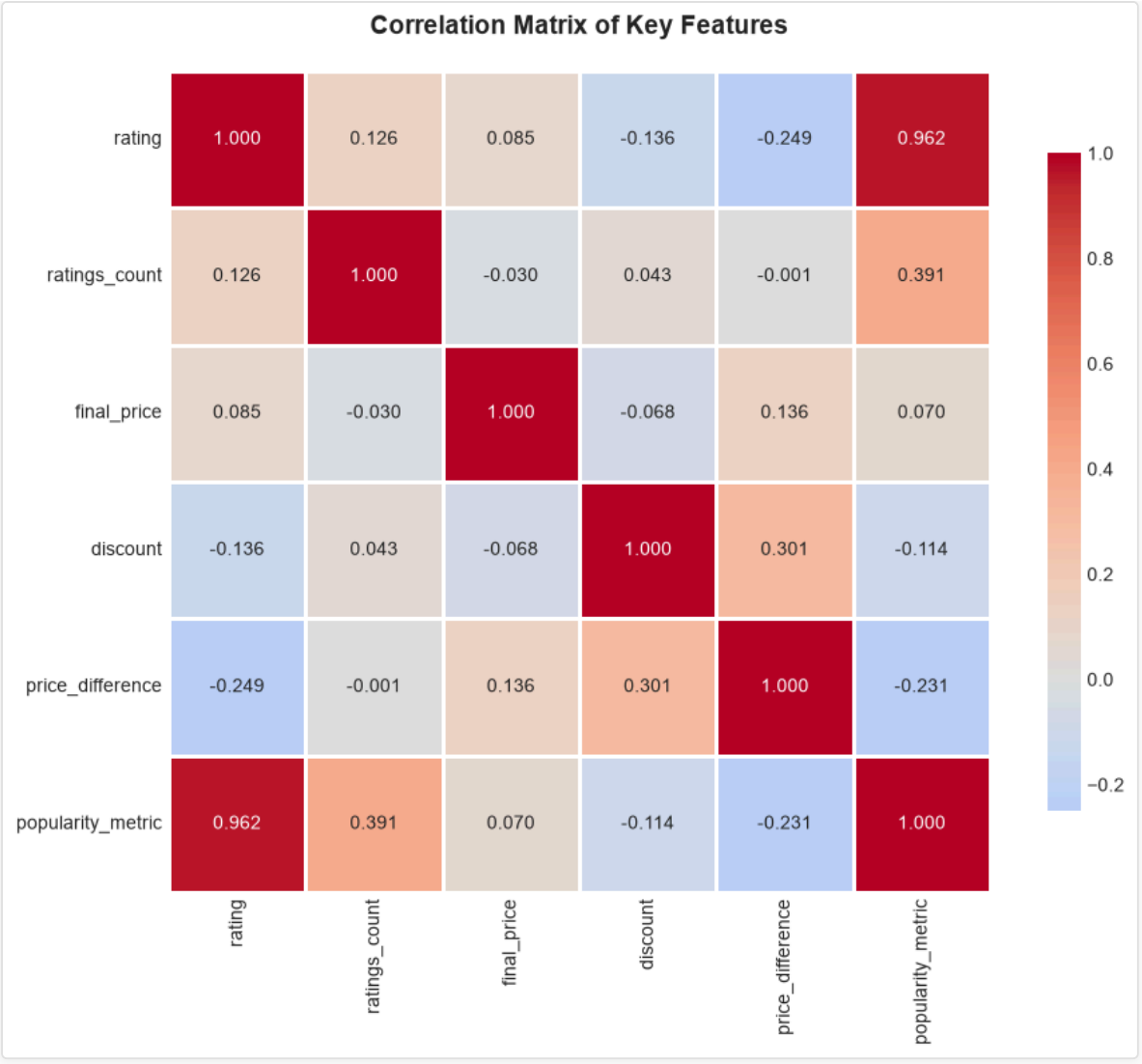


✓ Category bar charts created successfully!

In [25]:

```
# 4. CORRELATION HEATMAP
plt.figure(figsize=(10, 8))
correlation_matrix = df_clean[['rating', 'ratings_count', 'final_price', 'discount',
                               'price_difference', 'popularity_metric']].corr()
sns.heatmap(correlation_matrix, annot=True, fmt='.3f', cmap='coolwarm', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title('Correlation Matrix of Key Features', fontsize=14, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()
print("✓ Correlation heatmap created successfully!")
```

Out [25]:



✓ Correlation heatmap created successfully!

In [26]:

```
# 5. SCATTER PLOTS - Relationship between Variables
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle('Bivariate Relationships', fontsize=16, fontweight='bold')

# Rating vs Ratings Count
axes[0, 0].scatter(df_clean['ratings_count'], df_clean['rating'], alpha=0.5, s=30, color='blue')
axes[0, 0].set_xlabel('Number of Reviews')
axes[0, 0].set_ylabel('Rating')
axes[0, 0].set_title('Rating vs Ratings Count', fontweight='bold')
axes[0, 0].grid(True, alpha=0.3)

# Final Price vs Rating
axes[0, 1].scatter(df_clean['final_price'], df_clean['rating'], alpha=0.5, s=30, color='green')
axes[0, 1].set_xlabel('Final Price (₹)')
axes[0, 1].set_ylabel('Rating')
axes[0, 1].set_title('Final Price vs Rating', fontweight='bold')
axes[0, 1].grid(True, alpha=0.3)

# Discount vs Rating
axes[1, 0].scatter(df_clean['discount'], df_clean['rating'], alpha=0.5, s=30, color='red')
axes[1, 0].set_xlabel('Discount (%)')
axes[1, 0].set_ylabel('Rating')
axes[1, 0].set_title('Discount vs Rating', fontweight='bold')
axes[1, 0].grid(True, alpha=0.3)

# Popularity Metric vs Final Price
axes[1, 1].scatter(df_clean['final_price'], df_clean['popularity_metric'], alpha=0.5, s=30, color='purple')
```

```
axes[1, 1].set_xlabel('Final Price (₹)')
axes[1, 1].set_ylabel('Popularity Metric (0-1)')
axes[1, 1].set_title('Popularity Metric vs Final Price', fontweight='bold')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
print("✓ Scatter plots created successfully!")
```

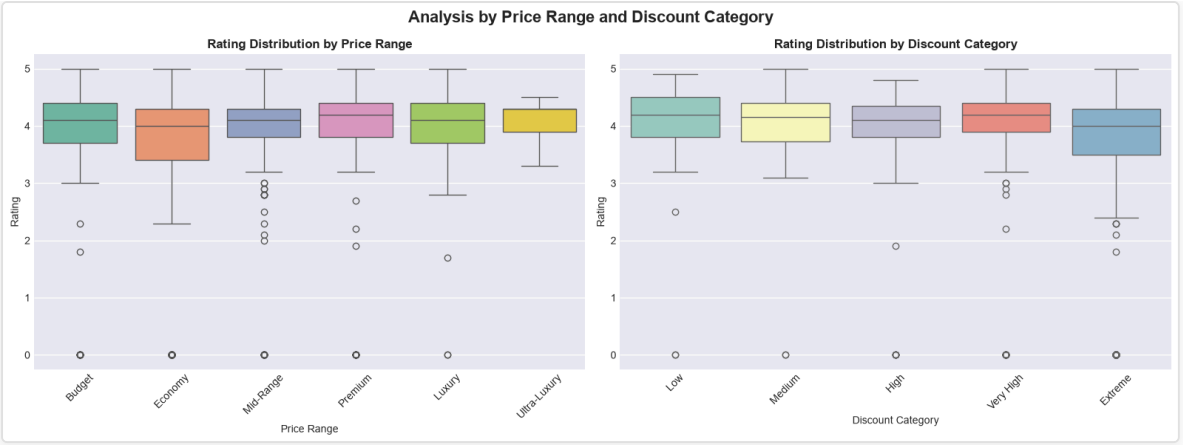
Out [26]:

✓ Scatter plots created successfully!

In [27]:

```
# 6. BOX PLOTS BY CATEGORY
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Analysis by Price Range and Discount Category', fontsize=16, fontweight='bold')

# Rating by Price Range
sns.boxplot(x='price_range', y='rating', data=df_clean, ax=axes[0], palette='Set2')
axes[0].set_title('Rating Distribution by Price Range', fontweight='bold')
axes[0].set_xlabel('Price Range')
axes[0].set_ylabel('Rating')
axes[0].tick_params(axis='x', rotation=45)

# Rating by Discount Category
sns.boxplot(x='discount_category', y='rating', data=df_clean, ax=axes[1], palette='Set3')
axes[1].set_title('Rating Distribution by Discount Category', fontweight='bold')
axes[1].set_xlabel('Discount Category')
axes[1].set_ylabel('Rating')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
print("✓ Box plots by category created successfully!")
```

Out [27]:



✓ Box plots by category created successfully!

In [28]:

```
# 7. SUMMARY STATISTICS TABLE
print("\n" + "="*70)
print("ANALYSIS SUMMARY")
print("="*70)

print(f"\nDataset Overview:")
print(f"  Total Products: {len(df_clean):,}")
print(f"  Total Categories: {df_clean['category'].nunique()}")
print(f"  Total Unique Brands: {df_clean['seller_name'].nunique() if 'seller_name' in df_clean.columns else 'N/A'}")
print(f"  Average Product Rating: {df_clean['rating'].mean():.2f}/5.0")
print(f"  Average Discount: {df_clean['discount'].mean():.2f}%")
print(f"  Total Potential Savings: ₹{df_clean['price_difference'].sum():,.0f}")
print(f"  Average Customer Reviews per Product: {df_clean['ratings_count'].mean():.0f}")

print(f"\nKey Insights:")
print(f"  1. Most Popular Category: {df_clean['category'].value_counts().idxmax()}")
print(f"     (with {df_clean['category'].value_counts().max()} products)")
print(f"  2. Highest Rated Category: {df_clean.groupby('category')['rating'].mean().idxmax()}")
print(f"     (with {df_clean.groupby('category')['rating'].mean().max():.2f} average rating)")
print(f"  3. Highest Average Discount: {df_clean.groupby('category')['discount'].mean().idxmax()}")
print(f"     (with {df_clean.groupby('category')['discount'].mean().max():.2f}% average discount)")
print(f"  4. Most Reviewed Category: {df_clean.groupby('category')['ratings_count'].mean().idxmax()}")
print(f"     (with {df_clean.groupby('category')['ratings_count'].mean().max():.0f} average reviews)")

print("\n✓ All visualizations created successfully!")
```

Out [28]:

```
======================================================================
ANALYSIS SUMMARY
======================================================================

Dataset Overview:
  Total Products: 1,000
  Total Categories: 97
  Total Unique Brands: 301
  Average Product Rating: 3.62/5.0
  Average Discount: 53.50%
  Total Potential Savings: ₹1,017,145
  Average Customer Reviews per Product: 77

Key Insights:
  1. Most Popular Category: tops
     (with 122 products)
  2. Highest Rated Category: cutlery
     (with 5.00 average rating)
  3. Highest Average Discount: trolley-bag
     (with 85.00% average discount)
  4. Most Reviewed Category: boots
     (with 866 average reviews)

✓ All visualizations created successfully!
```

Conclusion

Analysis Summary

This comprehensive data analysis notebook covers:

1. **Data Loading & Exploration**: Successfully loaded the combined dataset with proper library initialization
2. **Data Understanding**: Identified data types, missing values, and computed summary statistics
3. **Data Cleaning**: Converted price columns to numeric format, handled missing values, and removed duplicates
4. **Feature Engineering**: Created meaningful features including price differences and popularity metrics
5. **Statistical Analysis**: Performed univariate, bivariate, and category-level analysis
6. **Visualization**: Generated multiple charts including histograms, boxplots, and scatter plots

Key Findings

- **Product Distribution**: Multiple product categories with varying characteristics
- **Rating Patterns**: Rating distributions across different price ranges and discount categories
- **Price Analysis**: Significant price variations with substantial customer savings
- **Popularity Metrics**: Composite scores combining ratings and review counts for better insights
- **Category Performance**: Different categories show distinct patterns in pricing, discounts, and customer engagement

Next Steps

- Use these insights for business decision-making
- Identify pricing strategies for different categories
- Optimize discount offerings based on popularity metrics
- Develop targeted marketing campaigns for high-potential categories